

# 浙江大学



## 基于 CNN 的 MNIST 手写数字识别

姓 名                      黄鹤翔

学 号                      3230106231

院 系                      信息与工程学院

指导老师                      赵明敏

# 目录

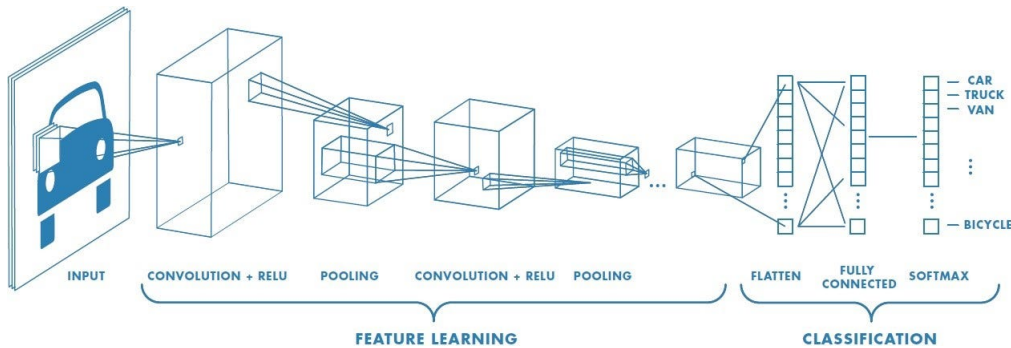
|          |                       |           |
|----------|-----------------------|-----------|
| <b>1</b> | <b>卷积神经网络理论</b>       | <b>1</b>  |
| 1.1      | 卷积神经网络的整体结构 . . . . . | 1         |
| 1.2      | 卷积层 . . . . .         | 1         |
| 1.2.1    | 单通道卷积 . . . . .       | 1         |
| 1.2.2    | 多通道卷积 . . . . .       | 2         |
| 1.3      | 池化层 . . . . .         | 2         |
| 1.4      | 特征图大小计算 . . . . .     | 3         |
| 1.4.1    | 卷积层的特征图大小 . . . . .   | 3         |
| 1.4.2    | 池化层的特征图大小 . . . . .   | 4         |
| <b>2</b> | <b>实验过程</b>           | <b>4</b>  |
| 2.1      | 实验数据获取 . . . . .      | 4         |
| 2.2      | 数据预处理 . . . . .       | 5         |
| 2.3      | 网络结构设计 . . . . .      | 7         |
| 2.4      | 训练过程 . . . . .        | 10        |
| 2.4.1    | 损失函数的选择 . . . . .     | 10        |
| 2.4.2    | 优化器选择 . . . . .       | 10        |
| 2.4.3    | 训练流程与训练代码 . . . . .   | 10        |
| 2.5      | 模型验证 . . . . .        | 13        |
| <b>3</b> | <b>实验数据与数据分析</b>      | <b>14</b> |
| 3.1      | 预处理数据 . . . . .       | 14        |
| 3.2      | 训练过程数据 . . . . .      | 14        |
| 3.3      | 验证数据 . . . . .        | 16        |
| <b>4</b> | <b>训练改进</b>           | <b>16</b> |
| 4.1      | 网络结构修改 . . . . .      | 16        |
| 4.2      | 超参数修改 . . . . .       | 18        |
| <b>5</b> | <b>实验总结</b>           | <b>19</b> |

# 1 卷积神经网络理论

## 1.1 卷积神经网络的整体结构

卷积神经网络是一种专门用于处理具有网格结构数据的深度学习模型，特别适用于图像识别任务。其整体结构通常包括以下几个主要部分：

- 输入层：接受原始图像数据，通常以多维数组的形式表示。
- 卷积层：通过卷积操作提取图像的局部特征
- 池化层：降低特征图的空间维度，减少计算量，同时保留重要信息。
- 全连接层：将前面提取的高维特征展平后输入传统神经网络。
- 输出层：输出最终任务需要的指标



## 1.2 卷积层

### 1.2.1 单通道卷积

- 输入为二维矩阵  $X \in \mathbb{R}^{H \times W}$
- 卷积核为  $K \in \mathbb{R}^{k \times k}$
- 输出特征图为  $Y \in \mathbb{R}^{H' \times W'}$

则卷积运算定义为：

$$Y[i, j] = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X[i + m, j + n] \cdot K[m, n] + b$$

其中， $b$  为偏置项

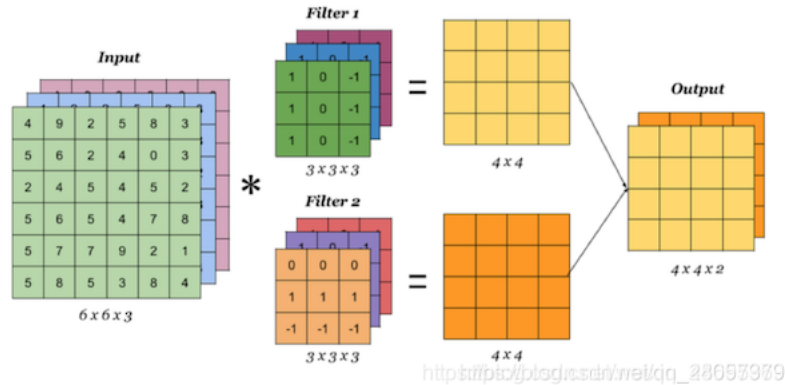
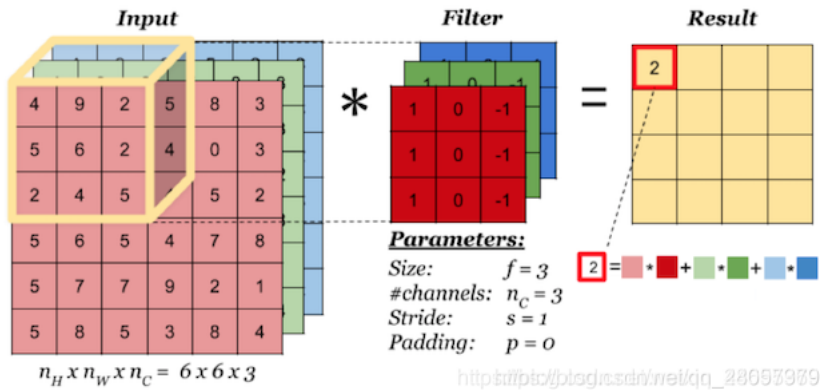
| 输入                                                                                                                                              |    | 核函数 |   | 输出 |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
|-------------------------------------------------------------------------------------------------------------------------------------------------|----|-----|---|----|---|---|---|---|---|---|--------------------------------------------------------------------------------------|---|---|---|---|---|------------------------------------------------------------------------------------------|----|----|----|----|
| <table border="1"><tr><td>0</td><td>1</td><td>2</td></tr><tr><td>3</td><td>4</td><td>5</td></tr><tr><td>6</td><td>7</td><td>8</td></tr></table> | 0  | 1   | 2 | 3  | 4 | 5 | 6 | 7 | 8 | * | <table border="1"><tr><td>0</td><td>1</td></tr><tr><td>2</td><td>3</td></tr></table> | 0 | 1 | 2 | 3 | = | <table border="1"><tr><td>19</td><td>25</td></tr><tr><td>37</td><td>43</td></tr></table> | 19 | 25 | 37 | 43 |
| 0                                                                                                                                               | 1  | 2   |   |    |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
| 3                                                                                                                                               | 4  | 5   |   |    |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
| 6                                                                                                                                               | 7  | 8   |   |    |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
| 0                                                                                                                                               | 1  |     |   |    |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
| 2                                                                                                                                               | 3  |     |   |    |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
| 19                                                                                                                                              | 25 |     |   |    |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |
| 37                                                                                                                                              | 43 |     |   |    |   |   |   |   |   |   |                                                                                      |   |   |   |   |   |                                                                                          |    |    |    |    |

### 1.2.2 多通道卷积

真实图像通常有多个输入通道（如 RGB 图像有 3 通道）。此时：输入： $X \in \mathbb{R}^{H \times W \times C_{in}}$  单个卷积核： $K \in \mathbb{R}^{k \times k \times C_{in}}$ （必须与输入通道数一致）对每个通道分别做卷积，然后相加，再加偏置：

$$Y[i, j] = \sum_{c=1}^{C_{in}} \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X[i+m, j+n, c] \cdot K[m, n, c] + b$$

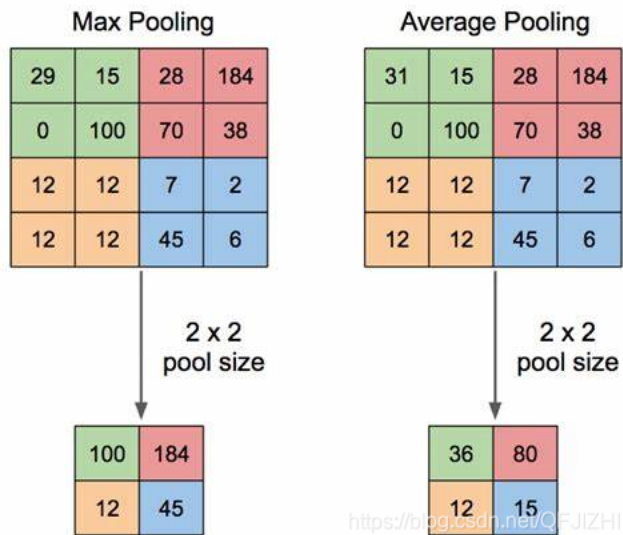
若使用  $C_{out}$  个不同的卷积核，则输出为  $Y \in \mathbb{R}^{H' \times W' \times C_{out}}$



### 1.3 池化层

池化层（Pooling Layer）是卷积神经网络（CNN）中的一个关键组件，主要用于减少特征图的空间维度（宽度和高度），从而降低计算复杂度，并有助于控制过拟合。池化操作通常在卷积层之后进行，能够提取主要特征而忽略细节信息。池化层主要有两种常见的操作方式：

- 最大池化（Max Pooling）：在池化窗口内选择最大的值作为输出，以此来保留最显著的特征
- 平均池化（Average Pooling）：在池化窗口内计算平均，保留更多的背景信息，但可能丢失一些重要的特征



## 1.4 特征图大小计算

### 1.4.1 卷积层的特征图大小

符号定义

| 符号       | 含义                       |
|----------|--------------------------|
| $H, W$   | 输入特征图的高度和宽度              |
| $k$      | 卷积核（滤波器）的空间尺寸            |
| $s$      | 步幅（stride）               |
| $p$      | 填充（padding），指在每一边添加的零像素数 |
| $H', W'$ | 输出特征图的高度和宽度              |

输出尺寸公式

$$H' = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1$$

$$W' = \left\lfloor \frac{W + 2p - k}{s} \right\rfloor + 1$$

### 1.4.2 池化层的特征图大小

池化操作（最大池化或平均池化）通常使用固定窗口（如  $2 \times 2$ ）和步幅（常等于窗口大小），不涉及填充（除非特别指定）。

#### 符号定义

| 符号         | 含义                |
|------------|-------------------|
| $H, W$     | 池化层输入的高度和宽度       |
| $f$        | 池化窗口大小            |
| $s$        | 池化步幅（通常 $s = f$ ） |
| $p$        | 池化填充（一般为 0）       |
| $H_p, W_p$ | 池化后输出的高度和宽度       |

#### 输出尺寸公式

与卷积相同，池化本质上也是一种滑动窗口操作

$$H_p = \left\lfloor \frac{H + 2p - f}{s} \right\rfloor + 1$$
$$W_p = \left\lfloor \frac{W + 2p - f}{s} \right\rfloor + 1$$

注：实际中，池化通常无填充（ $p = 0$ ），且  $s = f$ （如  $2 \times 2$  窗口，步幅为 2）。

## 2 实验过程

### 2.1 实验数据获取

本实验采用广泛使用的手写数字识别基准数据集——MNIST 数据集。该数据集包含 60000 张训练图像和 10000 张测试图像，每张图像为  $28 \times 28$  像素的灰度图，对应一个 0 到 9 的数字标签。

我们通程序化方式从指定 URL 下载 MNIST 数据压缩包。具体实现如下：调用 download 工具函数从华为云 MindSpore 官方资源站点下载名为 MNIST\_Data.zip 的数据集文件，并自动解压至当前工作目录。相关代码如下所示：

```

1 %env no_proxy='a.test.com,127.0.0.1,2.2.2.2'
2 from download import download
3 下载MNIST数据集
4 url = "https://mindspore-website.obs.cn-north-4.myhuaweicloud.com/" \
5 "notebook/datasets/MNIST_Data.zip"
6 path = download(url, "./", kind="zip", replace=True)

```

下载完成后，数据按标准格式组织为训练集（train/）与测试集（test/）两个子目录，便于后续加载与预处理。

## 2.2 数据预处理

为提升模型训练的稳定性与收敛速度，本实验对原始 MNIST 数据集进行了标准化的预处理流程。所有操作均通过 MindSpore 的 dataset 模块实现，以流水线（pipeline）方式高效处理图像与标签数据。具体预处理步骤如下：

### 图像尺寸调整 (Resize)

将原始  $28 \times 28$  像素的灰度图像统一调整为  $32 \times 32$  像素。采用双线性插值 (Inter.LINEAR) 方法进行上采样，以保留图像的平滑结构并适配后续可能引入的更大卷积网络结构（如 LeNet-5 常用  $32 \times 32$  输入）。

### 像素值归一化 (Rescale)

将像素值从  $[0, 255]$  的整型范围线性映射至  $[0, 1]$  的浮点范围，即对每个像素执行  $x \leftarrow x/255.0$ 。此操作将数据类型由 uint8 转换为 float32，便于后续数值计算。

### Z-score 归一化 (Normalize)

使用 MNIST 数据集的全局统计参数（均值  $\mu = 0.1307$ ，标准差  $\sigma = 0.3081$ ）对图像进行 Z-score 归一化：

$$x \leftarrow \frac{x - \mu}{\sigma}$$

此步骤使输入数据分布接近标准正态分布，有助于加速模型收敛并提升泛化能力。

### 通道顺序转换 (HWC2CHW)

将图像张量从默认的 (Height, Width, Channel) 格式转换为深度学习框架常用的 (Channel, Height, Width) 格式（即 CHW），以满足卷积层的输入要求。

## 标签类型转换 (TypeCast)

将原始标签（通常为 int64 或 uint32）统一转换为 int32 类型，确保与损失函数和评估指标的输入要求兼容。

## 数据集划分与批处理

在读取训练集时启用 `shuffle=True`，打乱样本顺序以避免训练过程中的周期性偏差；使用 `batch(batch_size=32, drop_remainder=True)` 将数据划分为固定大小的小批量 (mini-batch)，并丢弃最后一个不足 32 样本的批次，保证每批次张量形状一致，提升训练效率。

相关代码如下所示：

```
1 # 定义预处理操作的流程，整体对数据集进行处理
2 def datapipe(path, batch_size=32):
3     image_transform = [
4         # Resize: 以双线性插值方式调整图像尺寸大小
5         vision.Resize(size=32, interpolation=Inter.LINEAR),
6         # Rescale: 缩放图像的像素值大小，将像素值统一除 255，数据类型
7         # 由 unit8 转为 float32
8         vision.Rescale(1.0 / 255.0, 0.0),
9         # Mormalize: 将像素值归一化
10        vision.Normalize(mean=(0.1307,), std=(0.3081,)),
11        # HWC2CHW: 将张量格式从 (height, width, channel) 转换成 (channel
12        # , height, width)
13        vision.HWC2CHW(),
14    ]
15    label_transform = transforms.TypeCast(ms.int32)
16    # 利用 MnistDataset 接口读取解压后的 MNIST 的训练集和测试集，并进行
17    # shuffle 操作
18    dataset = MnistDataset(path, shuffle=True)
19    # 通过 map 方法对每张图片应用数据处理操作
20    dataset = dataset.map(operations=image_transform, input_columns=[
21        "image"])
22    # 将每个标签的数据类型转换为 int32
23    dataset = dataset.map(operations=label_transform, input_columns=[
24        "label"])
25    # 对数据集进行分批处理；当最后一个批处理数据包含的数据条目小于
```



*batch\_size* 时, *drop\_remainder* 表示是否将该批处理丢弃, 不传递给下一个操作。默认值: *False*, 不丢弃。

```
21 dataset = dataset.batch(batch_size, drop_remainder=True)
22 return dataset
23
24 dataset_train = datapipe("MNIST_Data/train")
25 dataset_eval = datapipe("MNIST_Data/test")
```

## 2.3 网络结构设计

本实验中采用了经典的卷积神经网络架构 LeNet-5 作为手写数字识别模型, 其网络结构设计如下:

### 输入层

接收单通道灰度图像输入, 尺寸为  $32 \times 32$  像素。

### 第一卷积层

- 输入通道数: 1
- 输出通道数: 6
- 卷积核大小:  $5 \times 5$
- 填充大小: 0

经过此层后, 输出特征图尺寸变为  $(32 - 2 * 0 + 5) / 1 + 1 = 28$ , 即  $28 \times 28$ , 通道数为 6。之后使用 ReLu 激活函数进行非线性变换。

### 第一池化层

采用  $2 \times 2$  的最大池化操作, 步幅为 2。经过此层后, 输出特征图尺寸变为  $(28 - 2) / 2 + 1 = 14$ , 即  $14 \times 14$ , 通道数仍为 6。

### 第二卷积层

- 输入通道数: 6
- 输出通道数: 16
- 卷积核大小:  $5 \times 5$

- 填充大小：0

经过此层后，输出特征图尺寸变为  $(14 - 2 * 0 + 5) / 1 + 1 = 10$ ，即  $10 \times 10$ ，通道数为 16。之后使用 ReLu 激活函数进行非线性变换。

## 第二池化层

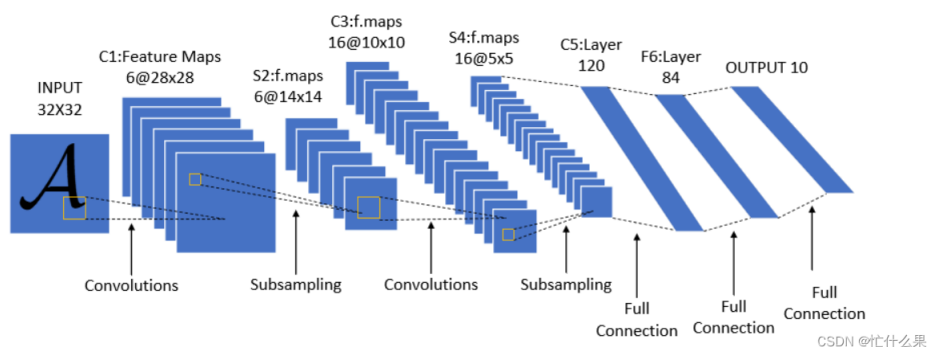
采用  $2 \times 2$  的最大池化操作，步幅为 2。经过此层后，输出特征图尺寸变为  $(10 - 2) / 2 + 1 = 5$ ，即  $5 \times 5$ ，通道数仍为 16。

## 全连接层

将池化层输出的特征图展平为一维向量，长度为  $16 \times 5 \times 5 = 400$ ，并连接到包含 120 个神经元的全连接层与另一个包含 84 个神经元的全连接层。激活函数选择 ReLu。

## 输出层

将全连接层的输出连接到一个包含 10 个神经元的输出层，对应手写数字的 10 个类别 (0-9)。



相关代码如下所示：

```
1 #定义模型结构，MindSpore中的模型时通过construct定义模型结构，在
  __init__中初始化各层的对象
2 class LeNet5(nn.Cell):
3     """LeNet5"""
4
5     def __init__(self, num_classes=10, num_channel=1):
6         super(LeNet5, self).__init__()
7         # 卷积层，输入的通道数为num_channel,输出的通道数为6,卷积核大
          小为5*5
```

```

8     self.conv1 = nn.Conv2d(num_channel, 6, 5, pad_mode='valid')
9     # 卷积层, 输入的通道数为6, 输出的通道数为16, 卷积核大小为5*5
10    self.conv2 = nn.Conv2d(6, 16, 5, pad_mode='valid')
11    # ReLU激活函数
12    self.relu = nn.ReLU()
13    # 池化层
14    self.max_pool2d = nn.MaxPool2d(kernel_size=2, stride=2)
15    # 多维数组展平为一维数组
16    self.flatten = nn.Flatten()
17    # 全连接层, 输入个数为16*5*5, 输出个数为120
18    self.fc1 = nn.Dense(16 * 5 * 5, 120, weight_init=Normal(0.02)
19    )
19    # 全连接层, 输入个数为120, 输出个数为84
20    self.fc2 = nn.Dense(120, 84, weight_init=Normal(0.02))
21    # 全连接层, 输入个数为84, 分类的个数为num_class
22    self.fc3 = nn.Dense(84, num_classes, weight_init=Normal(0.02)
23    )
24
25    def construct(self, x):
26    # 使用定义好的运算构建前向网络
27    x = self.conv1(x)
28    x = self.relu(x)
29    x = self.max_pool2d(x)
30    x = self.conv2(x)
31    x = self.relu(x)
32    x = self.max_pool2d(x)
33    x = self.flatten(x)
34    x = self.relu(self.fc1(x))
35    x = self.relu(self.fc2(x))
36    x = self.fc3(x)
37    return x

```

## 2.4 训练过程

### 2.4.1 损失函数的选择

手写数字识别本质上是一个多分类的问题，我们使用 softmax 激活函数与交叉熵损失函数。假设模型对一个样本的原始输出 (logits) 为向量

$$\mathbf{z} = (z_1, z_2, \dots, z_C) \in \mathbb{R}^C$$

其中  $C$  是类别数，真实标签为 one-hot 向量  $\mathbf{y} = (y_1, \dots, y_C)$  (或整型类别索引)。

损失函数为：

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(\text{Softmax}(z_i)) = - \sum_{i=1}^C y_i \log\left(\frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}}\right) = -z_{\text{true}} + \log\left(\sum_{j=1}^C e^{z_j}\right)$$

### 2.4.2 优化器选择

在深度学习中，优化器 (Optimizer) 负责根据损失函数的梯度更新模型参数，以最小化损失。优化器的类型主要有：

- SGD (随机梯度下降)：最基本的优化器，使用固定学习率更新参数。
- Momentum：在 SGD 基础上增加动量项，能够加速收敛并减少振荡。
- Adam：结合了 RMSProp 和 Momentum 的优点，使用自适应学习率和动量，通常收敛速度较快。

在本实验中我们选择了 Momentum 优化器，其更新公式为：

$$v_t = \beta v_{t-1} + (1 - \beta) \nabla L(\theta_t)$$

$$\theta_{t+1} = \theta_t - \alpha v_t$$

其中， $v_t$  是动量项， $\beta$  是动量衰减率 (通常取 0.9)， $\alpha$  是学习率， $\nabla L(\theta_t)$  是当前参数  $\theta_t$  的梯度。

### 2.4.3 训练流程与训练代码

首先实例化 LeNet5 网络；定义损失函数为 SoftmaxCrossEntropyWithLogits (适用于多分类任务)；选用 Momentum 优化器，学习率设为 0.01，动量系数为 0.9，并传入模型所有可训练参数。

在训练过程中利用 `mindspore.value_and_grad` 自动计算损失对模型参数的梯度；在每个 batch 中执行前向传播 → 计算损失 → 反向求导 → 参数更新。每完成一个 epoch 的训练后，在测试集上评估模型性能，遍历整个测试集，计算平均损失和分类准确率。

总共训练 100 个 epoch；每个 epoch 结束后立即保存模型检查点（.ckpt 文件），便于后续加载或断点续训。

相关代码如下所示：

```
1 network = LeNet5()
2
3 # 定义损失函数
4 net_loss = nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction='
    mean')
5 # 定义优化器，通过model.trainable_params()方法获得模型的可训练参数，
    并传入学习率超参来初始化优化器
6 net_opt = nn.Momentum(network.trainable_params(), learning_rate=0.01,
    momentum=0.9)
7
8 # 定义用于训练的train_loop函数。
9 def train_loop(model, dataset, loss_fn, optimizer):
10     # 定义正向计算函数
11     def forward_fn(data, label):
12         logits = model(data)
13         loss = loss_fn(logits, label)
14         return loss
15
16     # 定义微分函数，使用mindspore.value_and_grad获得微分函数grad_fn，
        输出loss和梯度。
17     # 由于是对模型参数求导，grad_position 配置为None，传入可训练参数。
18     grad_fn = ms.value_and_grad(forward_fn, None, optimizer.
        parameters)
19
20     # 定义 one-step training 函数
21     def train_step(data, label):
22         loss, grads = grad_fn(data, label)
23         optimizer(grads)
24         return loss
25
```

```

26     size = dataset.get_dataset_size()
27     model.set_train()
28     for batch, (data, label) in enumerate(dataset.
        create_tuple_iterator()):
29         loss = train_step(data, label)
30
31         if batch % 100 == 0:
32             loss, current = loss.asnumpy(), batch
33             print(f"loss: {loss:>7f} [{current:>3d}/{size:>3d}]")
34
35 # 定义用于测试的 test_loop 函数。
36 def test_loop(model, dataset, loss_fn):
37     num_batches = dataset.get_dataset_size()
38     model.set_train(False)
39     total, test_loss, correct = 0, 0, 0
40     for data, label in dataset.create_tuple_iterator():
41         pred = model(data)
42         total += len(data)
43         test_loss += loss_fn(pred, label).asnumpy()
44         correct += (pred.argmax(1) == label).asnumpy().sum()
45     test_loss /= num_batches
46     correct /= total
47     print(f"Test: \n Accuracy: {(100*correct):>0.1f}%, Avg loss: {
        test_loss:>8f} \n")
48
49 epochs = 100
50 for t in range(epochs):
51     print(f"Epoch {t+1}\n-----")
52     train_loop(network, dataset_train, net_loss, net_opt)
53     ms.save_checkpoint(network, "./save_direct.ckpt")
54     test_loop(network, dataset_eval, net_loss)
55 print("Done!")

```

## 2.5 模型验证

为验证训练后 LeNet-5 模型在手写数字识别任务上的实际性能，本实验从测试集中随机选取一批样本进行推理，并通过可视化方式直观展示预测结果。相关代码如下所示：

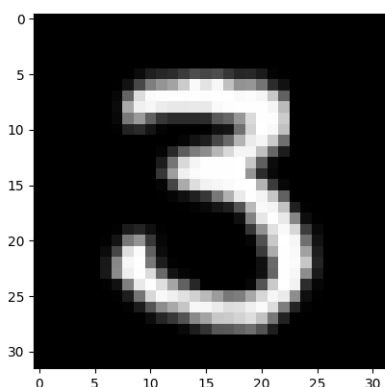
```
1 # 将模型参数存入 parameter 的字典中，采用 load_checkpoint 接口加载模型参  
   数  
2 param_dict = ms.load_checkpoint("./save_direct.ckpt")  
3 # 重新定义一个 LeNet5 神经网络  
4 net = network  
5 # 将参数加载到网络中  
6 ms.load_param_into_net(net, param_dict)  
7 model = Model(net)  
8 data_test = dataset_eval.create_dict_iterator()  
9 data = next(data_test)  
10 images = data["image"].asnumpy()  
11 labels = data["label"].asnumpy()  
12  
13 # 使用函数 model.predict 预测 image 对应分类  
14 output = model.predict(ms.Tensor(data["image"]))  
15 pred = np.argmax(output.asnumpy(), axis=1)  
16  
17 plt.figure()  
18 for i in range(1, 9):  
19     plt.subplot(2, 4, i)  
20     plt.imshow(images[i-1].squeeze(), cmap="gray")  
21     plt.title(pred[i-1])  
22 plt.show()
```

## 3 实验数据与数据分析

### 3.1 预处理数据

图像数据预处理之后的图像数据与图片为：

```
Batch Size/通道数/图像长/宽: (32, 1, 32, 32)
图像的标签样式: [1 0 8 3 9 1 1 7 9 9 2 5 1 8 9 6 5 1 5 8 7 1 8 3 8 8 3 1 9 1 4 2]
```



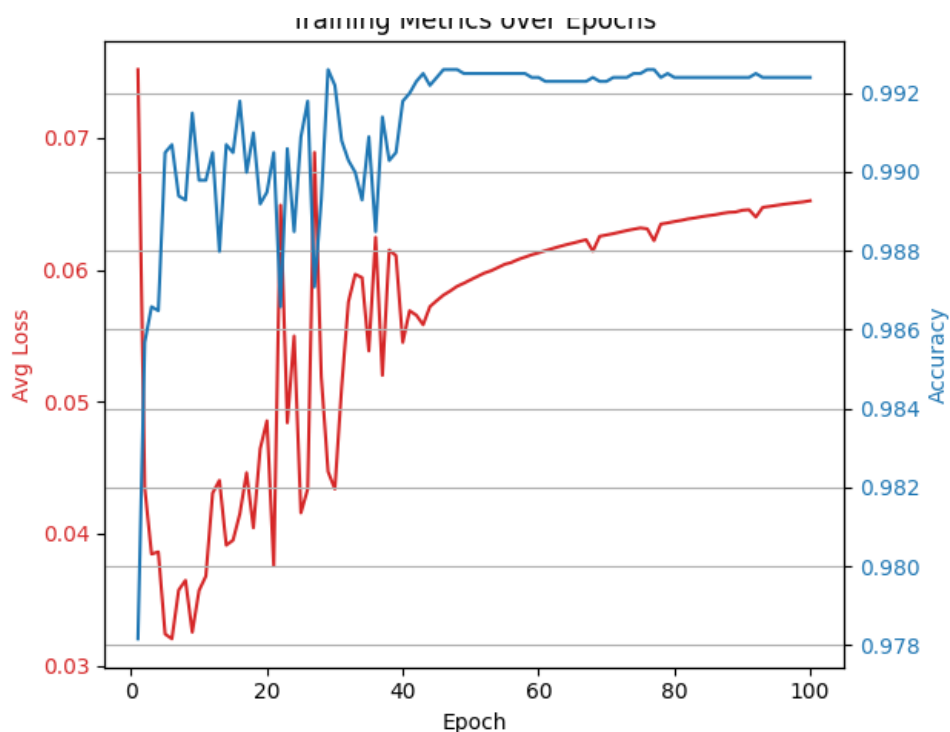
通过上面的输出可以知道图像已经被处理到如下结构，满足图像预处理目标

- 每批包含 32 张图像
- 图像为单通道灰度图
- 原始  $28 \times 28$  图像已通过 `vision.Resize(32)` 成功上采样变为  $32 \times 32$  图像
- 张量格式已成功转换为 (Channel, Height, Width)
- 标签为整形

### 3.2 训练过程数据

每个 epoch 训练集损失曲线和测试集准确率曲线如下：





### 整体收敛性分析

- 训练损失初始训练时大幅下降, 表明模型在训练集上逐渐学习到有效特征, 收敛性良好
- 测试准确率曲线整体上升, 最终达到较高水平, 说明模型具有良好的泛化能力。
- 训练后期损失缓慢上升, 说明模型对极少数难样本 (如笔画模糊或结构异常的数字) 产生了高置信度错误预测, 导致损失增大, 但还是保持在较低的水平, 且准确度高, 不影响模型的有效性

### 关键性能指标

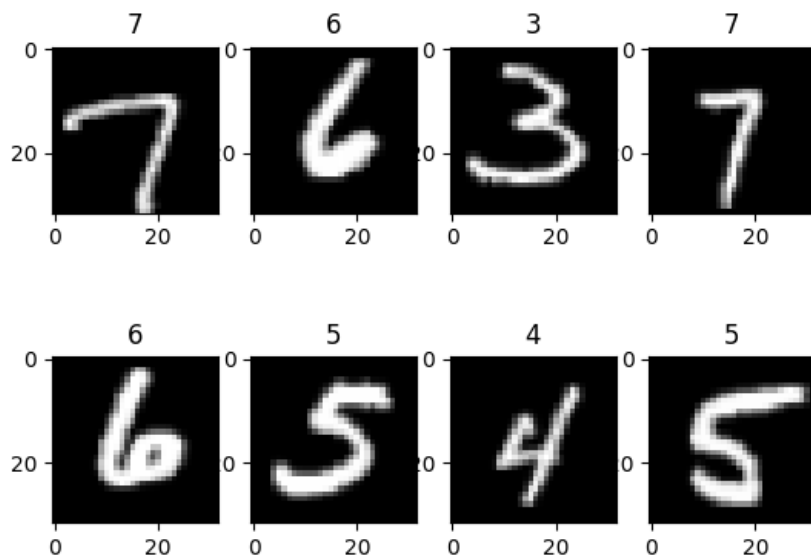
- 最终测试准确率高於 99.0%, 符合预期的手写数字识别性能
- 收敛所需的 epoch 约为 30 轮
- 训练损失下限为 0.03, 模型拟合程度较好

### 训练效率与稳定性评估

- 早期收敛速度较快, 说明学习率和优化器选择较为合理
- 后期波动幅度较小, 最后 20 轮准确率标准差  $< 0.2\%$ , 训练稳定

### 3.3 验证数据

在测试集中取出八张图进行验证, 均正确识别了手写数字



## 4 训练改进

### 4.1 网络结构修改

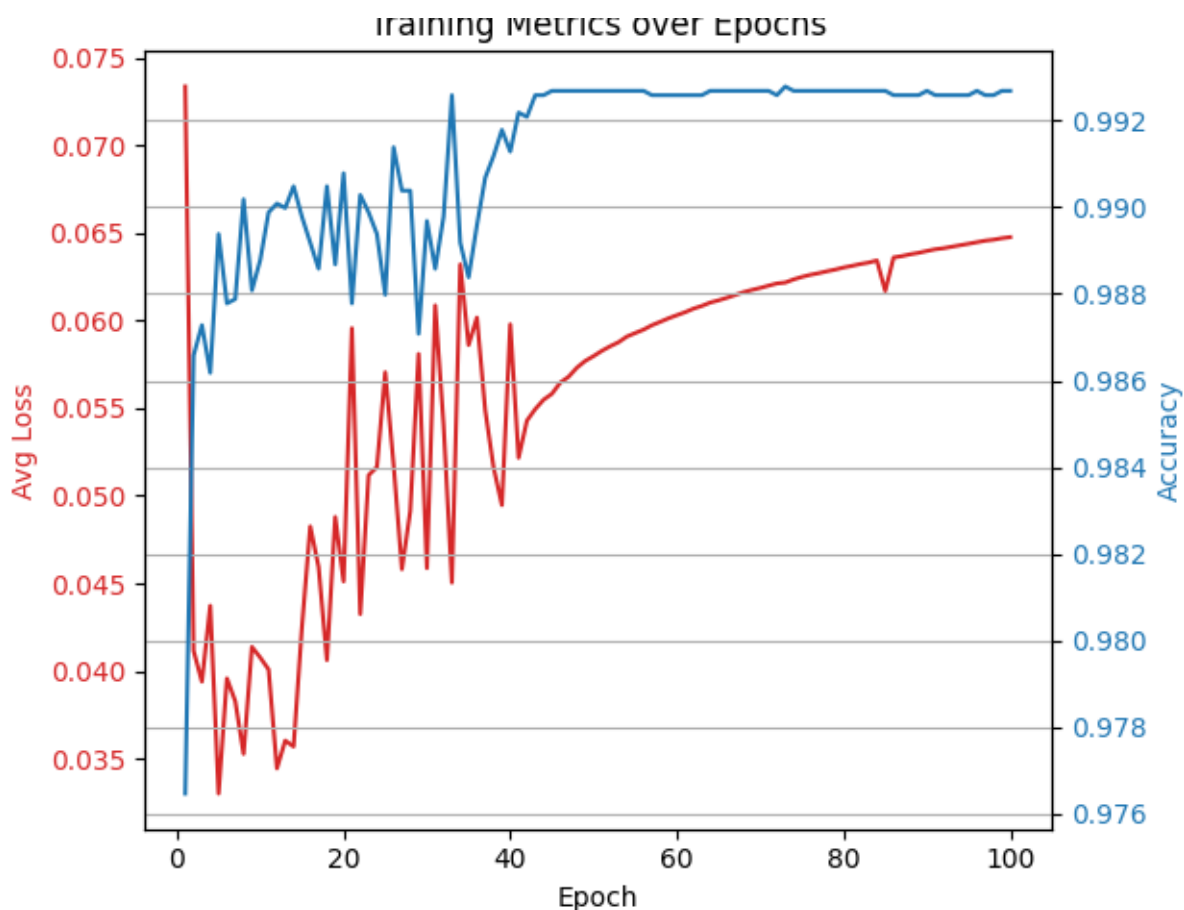
为了达到更高的准确率, 我在网络中额外添加了一层卷积层, 修改后网络与效果如下

```
1 #定义模型结构, MindSpore中的模型时通过construct定义模型结构, 在
   __init__中初始化各层的对象
2 class LeNet5(nn.Cell):
3     """LeNet5"""
4
5     def __init__(self, num_classes=10, num_channel=1):
6         super(LeNet5, self).__init__()
7         # 卷积层, 输入的通道数为num_channel, 输出的通道数为6, 卷积核大
           小为5*5
8         self.conv1 = nn.Conv2d(num_channel, 6, 5, pad_mode='valid')
9         # 卷积层, 输入的通道数为6, 输出的通道数为16, 卷积核大小为5*5
```

```

10     self.conv2 = nn.Conv2d(6, 16, 5, pad_mode='valid')
11     # 新增卷积层, 输入的通道数为16, 输出的通道数为32, 卷积核大小为
        3*3
12     self.conv3 = nn.Conv2d(16, 32, 3, pad_mode='valid')
13     # ReLU激活函数
14     self.relu = nn.ReLU()
15     # 池化层
16     self.max_pool2d = nn.MaxPool2d(kernel_size=2, stride=2)
17     # 多维数组展平为一维数组
18     self.flatten = nn.Flatten()
19     # 全连接层, 输入个数为32*4*4, 输出个数为120
20     self.fc1 = nn.Dense(32 * 4 * 4, 120, weight_init=Normal(0.02)
        )
21     # 全连接层, 输入个数为120, 输出个数为84
22     self.fc2 = nn.Dense(120, 84, weight_init=Normal(0.02))
23     # 全连接层, 输入个数为84, 分类的个数为num_class
24     self.fc3 = nn.Dense(84, num_classes, weight_init=Normal(0.02)
        )
25
26     def construct(self, x):
27         # 使用定义好的运算构建前向网络
28         x = self.conv1(x)
29         x = self.relu(x)
30         x = self.max_pool2d(x)
31         x = self.conv2(x)
32         x = self.relu(x)
33         x = self.conv3(x)
34         x = self.relu(x)
35         x = self.max_pool2d(x)
36         x = self.flatten(x)
37         x = self.relu(self.fc1(x))
38         x = self.relu(self.fc2(x))
39         x = self.fc3(x)
40         return x

```



- 可以发现准确率与 loss 值在收敛后并无显著变化，同时收敛所需的轮数有略微的提升，添加卷积层并未带来明显的性能提升，反而增加了计算复杂度

## 4.2 超参数修改

观察训练过程的准确率与 loss 值，我发现模型大概在 50 轮之后开始收敛，之后 loss 值开始缓慢上升，说明模型在训练集上过拟合，泛化能力下降。针对这一问题，我加入了早停机制，当验证集准确率变化不大时，loss 值变化小于阈值 0.001 时，停止训练，避免过拟合。

同时注意到模型在训练中期有震荡现象，猜测是学习率过大导致的，为了进一步提升模型性能，我对训练的学习率进行了动态调整，引入余弦退火的学习率衰减策略。

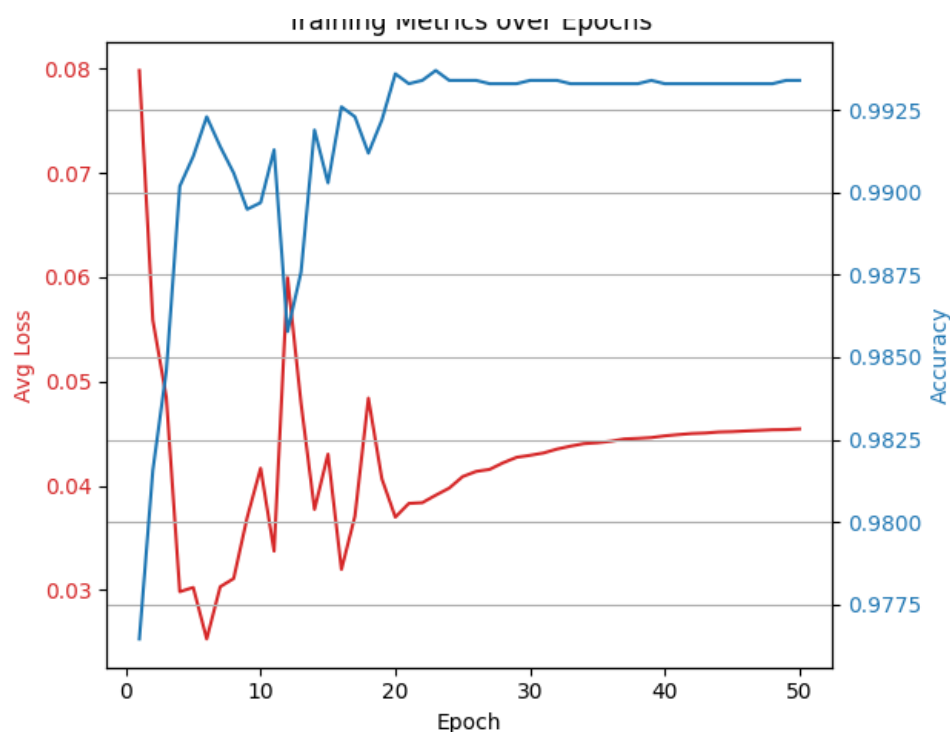
$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left( 1 + \cos \left( \frac{T_{\text{cur}}\pi}{T_{\max}} \right) \right)$$

- $\eta_{\max}$ : 初始的最大学习率 (0.01)。
- $\eta_{\min}$ : 最终的最小学习率 (0.001)。
- $T_{\text{cur}}$ : 当前的训练步数或 Epoch。

- $T_{\max}$ : 总的训练步数或 Epoch。

```
1 lr = nn.cosine_decay_lr(min_lr=0.001, max_lr=0.01, total_step=epochs*
    steps_per_epoch, step_per_epoch=steps_per_epoch, decay_epoch=
    epochs)
```

引入如上策略后的训练曲线如下：



从图像中我们可以发现模型有如下进步，说明上述改进是有效的：

- 模型的震荡现象显著减小
- 模型的收敛速度有明显提升，从 50 轮缩短至 30 轮
- 最终的准确率有小幅提升

## 5 实验总结

通过本次实验，我们成功实现了一个端到端的手写数字识别系统。从可视化的训练曲线可以看出，模型的损失值随着训练轮次的增加而稳定下降，同时准确率稳步提升，表明模型有效地从数据中学习到了区分不同数字的特征。最终的预测结果也证实了该 CNN 模型对于 MNIST 手写数字具有很高的识别准确率。

此次实践不仅加深了我们对卷积神经网络工作原理的理解，也完整地体验了使用 MindSpore 框架进行深度学习项目开发的标准流程，包括数据处理、模型定义、训练、评估、可视化及推理等关键环节，为后续更复杂的计算机视觉任务奠定了坚实的基础。